GenAI Code Comparison

EECS 348 - Software Engineering I | Assignment 4 | October 11, 2026

Student: Abdullah Hamad Al-khateeb

1. The two AI programs

I used **ChatGPT** through this ChatGPT conversation in a web browser, and **Google Gemini** through its website on the KU computer. I used the same prompt for both. I saved the original code from each one before working on the final version.

2. Exact prompt used for both

> Write a complete C++17 program for EECS 348 Assignment 4 that solves Sudoku puzzles using an object-oriented class and recursive depth-first search with backtracking. Read puzzle1.txt through puzzle5.txt from the current directory. Each file contains a 9x9 Sudoku grid with underscores representing empty cells. For each file, print the filename, original puzzle, and every valid solution, or "No solution found" if none exists. Follow Sudoku row, column, and 3x3 block rules. Include a main function, use standard C++ libraries, and make the code compatible with Linux g++. Provide the complete source code only.

The complete, original code from both AIs is included in the appendices. I did not use the improved version as either raw AI response.

3. Comparing the original code

Area	Gemini	ChatGPT
Approach	Uses a **Sudoku** class, a character grid, and recursive backtracking.	Uses a **SudokuSolver** class, an integer grid, and recursive backtracking.
Cell choice	Picks the first empty cell in row order.	Picks the empty cell with the fewest legal values (MRV).
Input checks	Opens the file and reads the grid. It does not check the starting numbers for contradictions.	Checks the file symbols and rejects contradictory starting numbers.
Output	Prints each original puzzle and every solution found.	Prints each original puzzle and every solution found.

4. Correctness tests

The two original programs and the updated final program were compiled and tested in a separate Linux environment using the five provided puzzle files. All three produced the same solution counts:

Input	Gemini	ChatGPT	Final
puzzle1.txt	1 solution	1 solution	1 solution
puzzle2.txt	1 solution	1 solution	1 solution
puzzle3.txt	1 solution	1 solution	1 solution
puzzle4.txt	No solution	No solution	No solution
puzzle5.txt	2 solutions	2 solutions	2 solutions

The first puzzle also matched the provided **solution1.txt**. Two extra checks were used because passing five test files does not mean a program handles every bad input correctly:

Extra test	Gemini	ChatGPT	Final
Completed grid with a duplicate digit	Incorrectly prints a solution	Rejects it	Rejects it
Extra non-whitespace symbol after a puzzle	Still processes the puzzle	Still processes the puzzle	Rejects the file

This is why I would not choose the Gemini code as-is. It works on the provided examples, but a filled grid with invalid numbers can be mistaken for a solution. ChatGPT handles that case, and the final version adds one more input check.

5. Execution time and space complexity

I ran each program 70 times in a separate Linux test with g++ **-O2** and output sent to **/dev/null**. These are median end-to-end times, including process startup. This was a comparison run, not a KU Cycle timing benchmark.

Gemini original	ChatGPT original	Updated final
1.74 ms	1.91 ms	1.71 ms

The differences are small enough that I would not claim a major speed improvement from this test. Gemini does less work per recursion level, while ChatGPT checks more empty cells to pick the best one. That extra work can help avoid unnecessary branches on harder puzzles.

All three programs use backtracking, so the **worst-case running time is exponential** in the number of empty cells (roughly $O(9^E)$, ignoring per-step checking). The recursion depth is at most the number of empty cells, so their main extra space is **O(E)**. ChatGPT also creates small vectors of candidates during the search. The updated version uses fixed-size arrays instead, avoiding those heap allocations.

6. Maintainability

Gemini is shorter and the basic steps are easy to follow. ChatGPT separates file loading, checking, searching, and printing into methods, which made it easier for me to work with. Both use classes, but ChatGPT has better handling for invalid starting puzzles. In the final version I added named constants and comments so the code is easier to read and explain.

7. Which program I chose and why

I chose **ChatGPT's program** as the starting point. It already checks the starting grid and uses the minimum-remaining-values method to choose an empty cell. I preferred those checks and the way the methods were organized over just picking whichever program looked shorter. Both programs solved the provided puzzles, so correctness on those five alone was not enough to decide.

8. Changes made to the selected program

Correctness: I added a check after reading the 81 grid symbols. If the file contains extra non-whitespace characters, the final program rejects it rather than silently ignoring them. The earlier check for contradictory given numbers was kept.

Execution time: I changed the temporary candidate lists from **std::vector** to **std::array** with a count. This removes repeated dynamic allocations while searching. It does not change the exponential worst case, and the timing results only show a small difference.

Space: The board is still a fixed 9x9 array and recursion still needs O(E) stack space. The candidate storage is now fixed-size and does not need separate heap allocations.

Maintainability: I replaced repeated numbers with named constants, added clear comments around the major blocks and important lines, and included a prologue with the author, inputs, output, creation date, and AI sources. The final file is **Assignment4.cpp**.

Final check: The revised program was compiled on KU Linux with **g++ -std=c++17 -Wall -Wextra -O2**. The run shown in the assignment workflow returned 1, 1, 1, 0, and 2 solutions for puzzles 1 through 5. The compile step showed no errors or warnings.

The following appendices contain the complete original source code produced by Gemini and ChatGPT. The final edited code is supplied separately as Assignment4.cpp.

Appendix A - Gemini original C++ code (1 of 3)

Raw AI response. All 142 lines are reproduced below, in the original order.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

class Sudoku {
private:
    char grid[9][9];

    bool isValid(int row, int col, char num) const {
        // Check row
        for (int j = 0; j < 9; ++j) {
            if (grid[row][j] == num) {
                return false;
            }
        }

        // Check column
        for (int i = 0; i < 9; ++i) {
            if (grid[i][col] == num) {
                return false;
            }
        }

        // Check 3x3 block
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[i + startRow][j + startCol] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    void solveRecursive(int& solutionCount) {
        int row = -1;
        int col = -1;
        bool isEmpty = false;

        // Find the next empty cell represented by '_'
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (grid[i][j] == '_') {
                    row = i;
                    col = j;
                    isEmpty = true;
                    break;
                }
            }
            if (isEmpty) {
```

Appendix A - Gemini original C++ code (2 of 3)

Continued from previous page.

```
                break;
            }
        }

        // If no empty cells remain, a valid solution is found
        if (!isEmpty) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printGrid();
            std::cout << "\n";
            return;
        }

        // Try digits '1' through '9'
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;
                solveRecursive(solutionCount);
                grid[row][col] = '_'; // Backtrack
            }
        }
    }

public:
    Sudoku() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                grid[i][j] = '_';
            }
        }
    }

    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> grid[i][j];
            }
        }
        return true;
    }

    void printGrid() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << grid[i][j] << (j == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
    }
```

Appendix A - Gemini original C++ code (3 of 3)

Continued from previous page.

```
    void solve(int& solutionCount) {
        solutionCount = 0;
        solveRecursive(solutionCount);
    }
};

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        Sudoku sudoku;

        std::cout << "Filename: " << filename << "\n";
        if (!sudoku.loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename << "\n";
            std::cout << "----------------------------------------\n\n";
            continue;
        }

        std::cout << "Original Puzzle:\n";
        sudoku.printGrid();
        std::cout << "\n";

        int solutionCount = 0;
        sudoku.solve(solutionCount);

        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        }

        std::cout << "----------------------------------------\n\n";
    }

    return 0;
}
```

Appendix B - ChatGPT original C++ code (1 of 3)

Raw AI response. All 160 lines are reproduced below, in the original order.

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class SudokuSolver {
private:
    std::array<std::array<int, 9>, 9> board{};

    bool canPlace(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (board[row][i] == value || board[i][col] == value) {
                return false;
            }
        }

        int boxRow = (row / 3) * 3;
        int boxCol = (col / 3) * 3;
        for (int r = boxRow; r < boxRow + 3; ++r) {
            for (int c = boxCol; c < boxCol + 3; ++c) {
                if (board[r][c] == value) {
                    return false;
                }
            }
        }
        return true;
    }

    bool givenCellsAreValid() {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                int value = board[row][col];
                if (value == 0) {
                    continue;
                }
                board[row][col] = 0;
                bool valid = canPlace(row, col, value);
                board[row][col] = value;
                if (!valid) {
                    return false;
                }
            }
        }
        return true;
    }

    void search(long long &solutionCount) {
        int selectedRow = -1;
        int selectedCol = -1;
        std::vector<int> selectedCandidates;
        int fewestCandidates = 10;

        for (int row = 0; row < 9; ++row) {
```

Appendix B - ChatGPT original C++ code (2 of 3)

Continued from previous page.

```
            for (int col = 0; col < 9; ++col) {
                if (board[row][col] != 0) {
                    continue;
                }

                std::vector<int> candidates;
                for (int number = 1; number <= 9; ++number) {
                    if (canPlace(row, col, number)) {
                        candidates.push_back(number);
                    }
                }
                if (candidates.empty()) {
                    return;
                }
                if (static_cast<int>(candidates.size()) < fewestCandidates) {
                    fewestCandidates = static_cast<int>(candidates.size());
                    selectedRow = row;
                    selectedCol = col;
                    selectedCandidates = candidates;
                }
            }
        }

        if (selectedRow == -1) {
            ++solutionCount;
            std::cout << "Solution " << solutionCount << ":\n";
            printBoard();
            std::cout << '\n';
            return;
        }

        for (int number : selectedCandidates) {
            board[selectedRow][selectedCol] = number;
            search(solutionCount);
            board[selectedRow][selectedCol] = 0;
        }
    }

public:
    bool load(const std::string &filename) {
        std::ifstream input(filename);
        if (!input) {
            return false;
        }

        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                char symbol;
                if (!(input >> symbol) ||
                    (symbol != '_' && (symbol < '1' || symbol > '9'))) {
                    return false;
                }
                board[row][col] = (symbol == '_') ? 0 : symbol - '0';
            }
```

Appendix B - ChatGPT original C++ code (3 of 3)

Continued from previous page.

```
        }
        return true;
    }

    void printBoard() const {
        for (int row = 0; row < 9; ++row) {
            for (int col = 0; col < 9; ++col) {
                if (col > 0) {
                    std::cout << ' ';
                }
                if (board[row][col] == 0) {
                    std::cout << '_';
                } else {
                    std::cout << board[row][col];
                }
            }
            std::cout << '\n';
        }
    }

    long long solveAll() {
        if (!givenCellsAreValid()) {
            return 0;
        }
        long long solutionCount = 0;
        search(solutionCount);
        return solutionCount;
    }
};

int main() {
    for (int puzzleNumber = 1; puzzleNumber <= 5; ++puzzleNumber) {
        std::string filename = "puzzle" + std::to_string(puzzleNumber) + ".txt";
        SudokuSolver solver;
        std::cout << "Filename: " << filename << '\n';

        if (!solver.load(filename)) {
            std::cout << "Unable to read puzzle file.\n\n";
            continue;
        }

        std::cout << "Original Puzzle:\n";
        solver.printBoard();
        std::cout << '\n';

        if (solver.solveAll() == 0) {
            std::cout << "No solution found\n";
        }
        std::cout << "\n";
    }
    return 0;
}
```